



Cinvestav



COCYTEN
CONSEJO DE CIENCIA Y TECNOLOGÍA
DEL ESTADO DE NAYARIT

LAB 5 Subtractor

Student: Alonso Emmanuel López Macías

Email's: usuario_cocytén_32_x@cinvestav.mx
alemlopezma@gmail.com

Student: Cesar Eduardo Inda Cenicerós

Email's: usuario_cocytén_32_x@cinvestav.mx
cesar.inda@iteso.mx

Full Professor: Dr. Martin Gonzalez Perez

15/04/2025

CINVESTAV, Av Del Bosque 1145, El Bajío 45017, Zapopan Jalisco.

COCYTEN, Boulevard Luis Donaldo Colosio. Ciudad Industrial, 63173 Tepic Nayarit.

Technical Report

® CINVESTAV - COCYTEN

Abstract: *FPGA Terasic*

Board – ALTERA Cyclone IV EP4CE115F29C7N

IDE – Quartus Prime 18.1

Boolean Algebra

Digital Gates & Logical Comparators

Git & GitHub Control Version

Keywords: *Subtractor, Symbol, Half Adder, Full Adder, Carry, Overflow, Carry & Two complement.*

Table of Contents

Table of Contents.....	ii
Table of Figures	iii
1.- INTRODUCTION	0
1.1.- Requirements.....	0
2.- FUNCTIONAL DESCRIPTION	1
2.1- Boolean Algebra Equations.....	4
2.2.- TrueTable.....	5
2.2.- Results	6
2.3.- GitHub Repository	7
3.- CHALLENGUES/ISSUES.....	8
4.- CONCLUSIONS.....	8

Table of Figures

Figure: 1 Bit Adder	1
Figure: 2 Symbol Creation.....	1
Figure: 3 Libraries.....	1
Figure: 4 8bitAdder_bdf.....	2
Figure: 5 8BitAdder_bsf.....	2
Figure: 6 Subtractor Circuit.....	2
Figure: 7 RTL Viewer	3
Figure: 8 PinPlanner.....	3
Figure: 9 TrueTable & Circuit 1BitSubtractor	5
Figure: 10 TrueTable 8Bit Subtractor	5
Figure: 11 Test1.....	6
Figure: 12 Test2.....	6
Figure: 13 Test3.....	6
Figure: 14 Test4.....	7
Figure: 15 GitHub Repository	7

1.- INTRODUCTION

To understand the concept of a subtractor, it is necessary to first grasp how a half adder and a full adder work, and how, through this functionality, we can derive both a subtractor and a restorer — essentially, a combinational logic circuit that performs the addition or subtraction of n-bit inputs. The purpose of this is to obtain the resulting difference or value from the given inputs.

In this context, the concept of overflow also comes into play. Overflow indicates the maximum value that a number can reach given the number of input bits. When this expected bit range is exceeded, an overflow occurs, signaling that the resulting value may be incorrect. For instance, in a microcontroller, a flag can be activated to indicate where an invalid or unexpected error occurred, allowing the system to trigger an error-handling routine or respond to the issue immediately.

1.1.- Requirements

Design an 8-bit subtractor circuit for two numbers (A and B) using two complements with the following characteristics:

- Both Inputs and outputs will have an 8-bit word with.
- The Inputs will be represented in two complement formats.
- The output must be in two complement formats.
- If an overflow occurs, it must be indicated with a single bit.

Step1- Design an adder circuit.

Step2- Add circuitry to perform subtraction and overflow detection.

Step3- Implement the circuit on the FPGA.

- Use switches SW[10] to SW[17] for input data A.
- Use switches SW[0] to SW[7] for input data B.
- Display input A using LEDs LEDR[10] to LEDR[17].
- Display input B using LEDs LEDR[0] to LEDR[7].
- Display the output data using LEDs LEDG[0] to LEDG[7].
- Display the overflow bit using LED LEDG[8].

2.- FUNCTIONAL DESCRIPTION

The first step was to create the BDF file of the circuit and then convert that circuit into a “**symbol**” in order to use it in the local or project libraries. First generate 1 bit adder in the project.

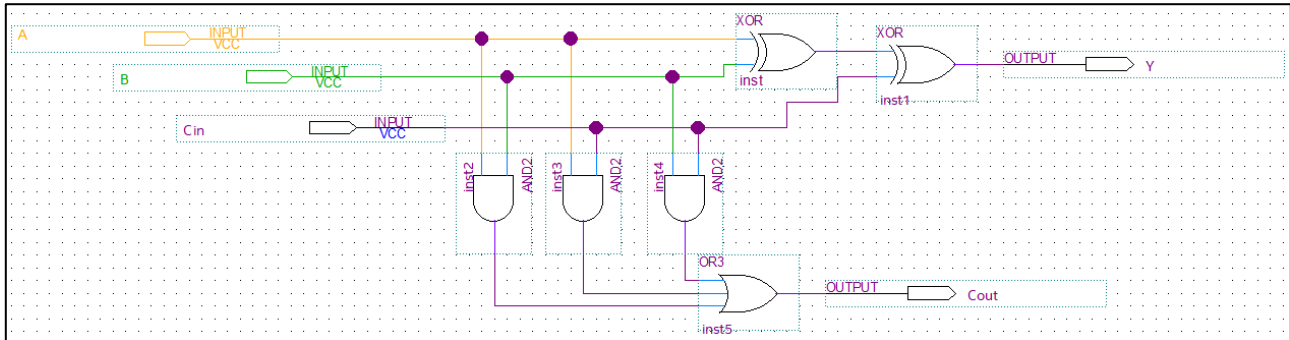


Figure: 1 Bit Adder

For create a symbol in the next option:

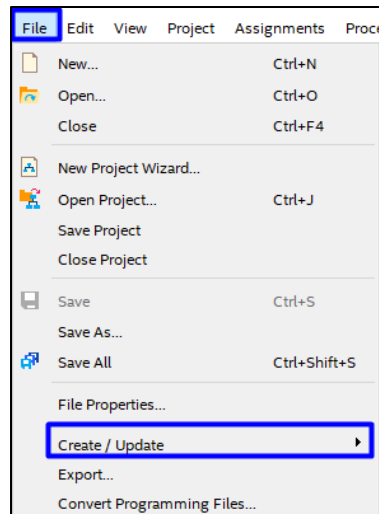


Figure: 2 Symbol Creation

This file will appear as a “.bsf” and can be found under “Libraries” → “Project” when placing symbols. The same case for create symbol for other schematics.

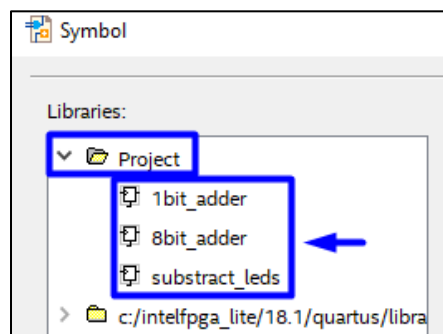


Figure: 3 Libraries

The next step was to create an 8-bit adder using this symbol, which would also be converted into a symbol during the process. This was done in order to use it as a final component of the subtractor.

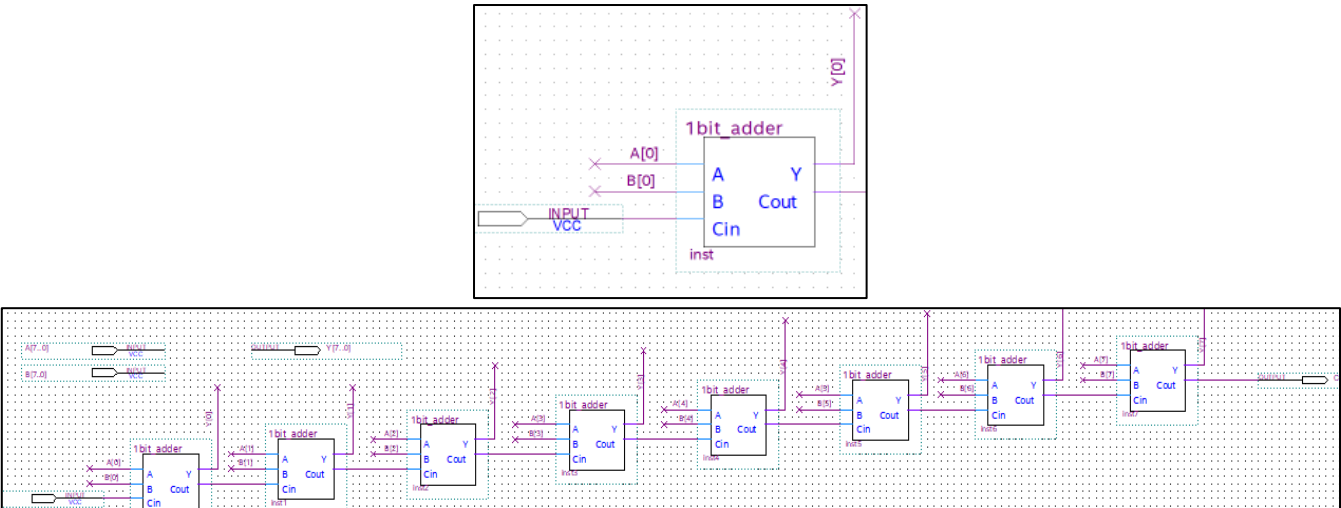


Figure: 4 8bitAdder_bdf

The final product as a symbol is 8bit_adder and then it is possible create the final circuit.

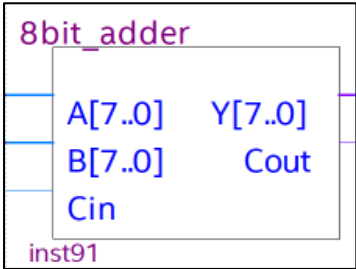


Figure: 5 8BitAdder_bsf

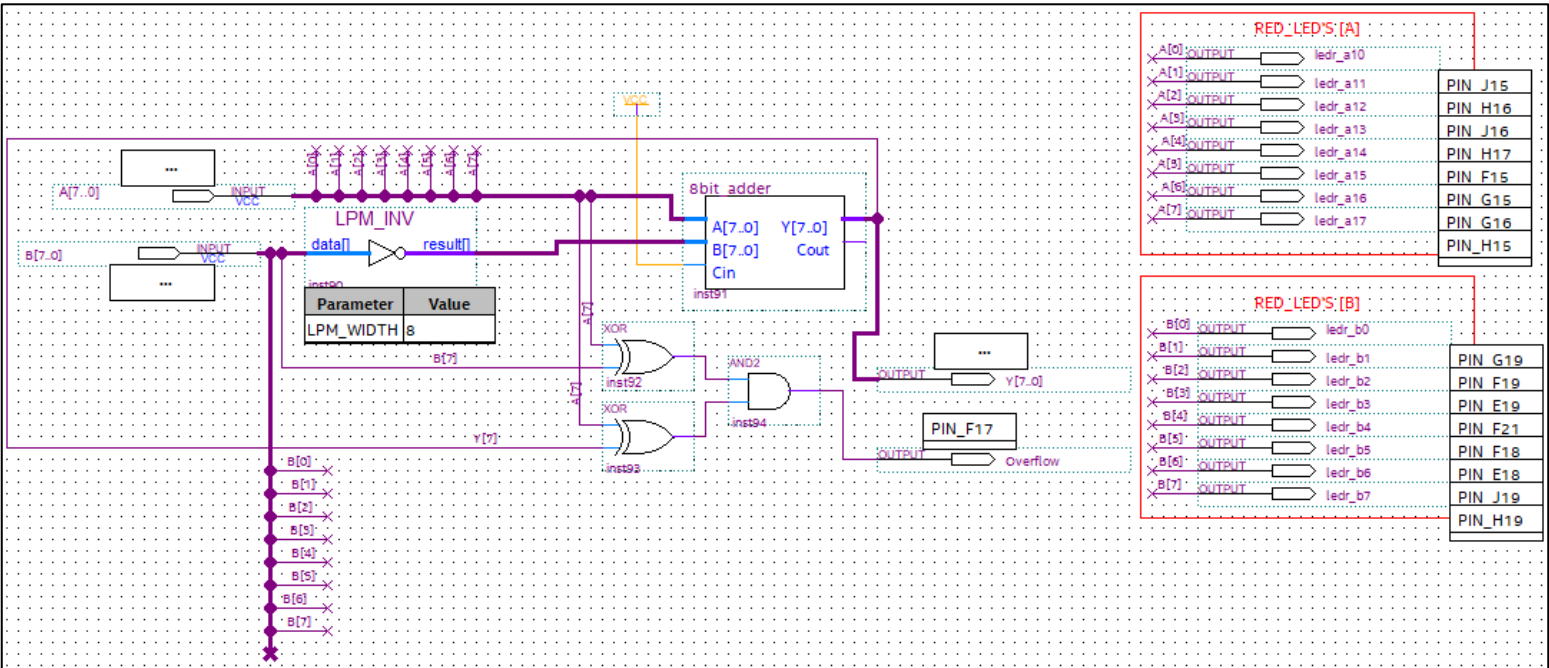


Figure: 6 Subtractor Circuit

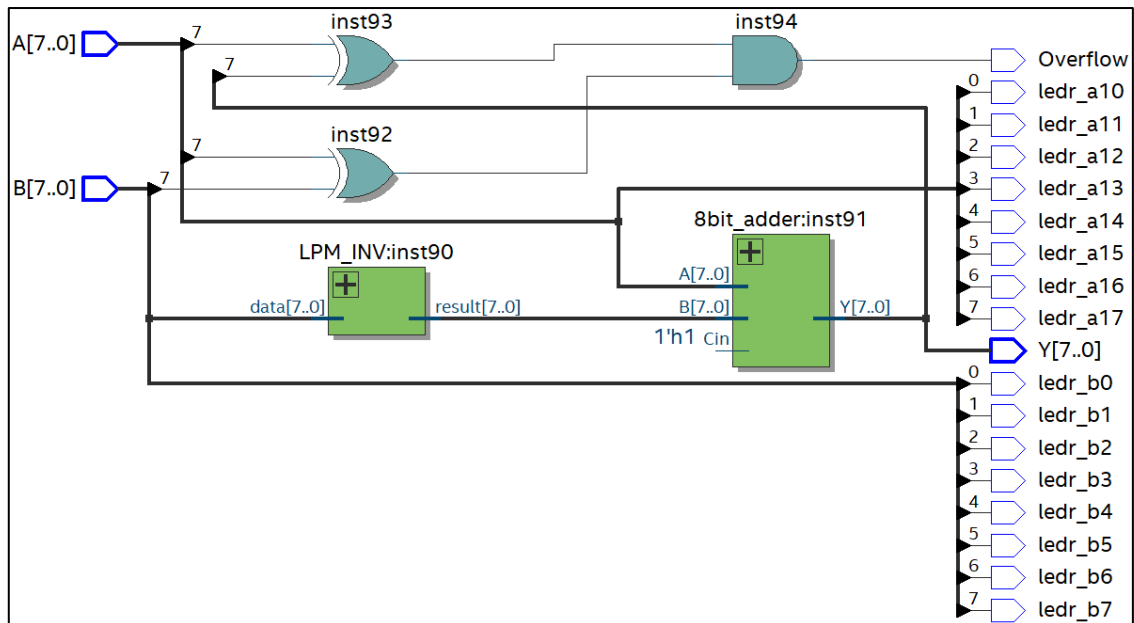
RTL VIEWER

Figure: 7 RTL Viewer

PIN PLANNER

in	A[7]	Input	PIN_Y23	out	ledr_a10	Output	PIN_J15
in	A[6]	Input	PIN_Y24	out	ledr_a11	Output	PIN_H16
in	A[5]	Input	PIN_AA22	out	ledr_a12	Output	PIN_J16
in	A[4]	Input	PIN_AA23	out	ledr_a13	Output	PIN_H17
in	A[3]	Input	PIN_AA24	out	ledr_a14	Output	PIN_F15
in	A[2]	Input	PIN_AB23	out	ledr_a15	Output	PIN_G15
in	A[1]	Input	PIN_AB24	out	ledr_a16	Output	PIN_G16
in	A[0]	Input	PIN_AC24	out	ledr_a17	Output	PIN_H15
in	B[7]	Input	PIN_AB26	out	ledr_b0	Output	PIN_G19
in	B[6]	Input	PIN_AD26	out	ledr_b1	Output	PIN_F19
in	B[5]	Input	PIN_AC26	out	ledr_b2	Output	PIN_E19
in	B[4]	Input	PIN_AB27	out	ledr_b3	Output	PIN_F21
in	B[3]	Input	PIN_AD27	out	ledr_b4	Output	PIN_F18
in	B[2]	Input	PIN_AC27	out	ledr_b5	Output	PIN_E18
in	B[1]	Input	PIN_AC28	out	ledr_b6	Output	PIN_J19
in	B[0]	Input	PIN_AB28	out	ledr_b7	Output	PIN_H19

out	Overflow	Output	PIN_F17
out	Y[7]	Output	PIN_G21
out	Y[6]	Output	PIN_G22
out	Y[5]	Output	PIN_G20
out	Y[4]	Output	PIN_H21
out	Y[3]	Output	PIN_E24
out	Y[2]	Output	PIN_E25
out	Y[1]	Output	PIN_E22
out	Y[0]	Output	PIN_E21

Figure: 8 PinPlanner

2.1- Boolean Algebra Equations

The subtractor uses the following components:

- 8-bit inverter (LPM_INV) to invert B.
- 8-bit adder (8bit_adder) that adds $A + \text{NOT}(B) + 1$ (i.e., two's complement).
- A logic section with XOR and AND gates to detect overflow.

This operation can be defined as:

$$A - B = A + (\bar{B} + 1).$$

Consideration for Overflow is:

$$\text{Overflow} = (A[7] \oplus B[7]) \cap (A[7] \oplus Y[7])$$

Brief Overflow:

- If the signs of A and B are different ($A[7] \oplus B[7] = 1$)
- And the sign of the result Y[7] is different from that of A, then overflow occurs.

Minterms are:

$$\text{Overflow} = (A7' \cdot B7 + A7 \cdot B7') \cdot (A7' \cdot Y7 + A7 \cdot Y7')$$

Equivalent Extend:

$$\text{Overflow} = \text{XOR}(A7, B7) \text{ AND } \text{XOR}(A7, Y7)$$

The considerations for the block symbol / function “full adder” are:

$$Y_i = A_i \oplus B_i \oplus C_i$$
$$C_{i+1} = A_i \cdot B_i + (A_i \oplus B_i) \cdot C_i$$

Were

“ A_i ” and B_i are the input bits

“ C_i ” is the carry-in

“ Y_i ” is the output bit

This implementation takes consideration for the symbols that were done previously in a circuit session logic, this allows the user or designer to use these symbols in any project for representing the combinational logic.

2.2.- TrueTable

For the purpose of the practice, it is necessary to understand the behavior of a subtractor for the product of 2-bit inputs, that is, $A = 1$ bit and $B = 1$ bit. Starting from this logic, one can interpret and understand its behavior through the truth table, including the concept of carry and how the idea of overflow is derived.

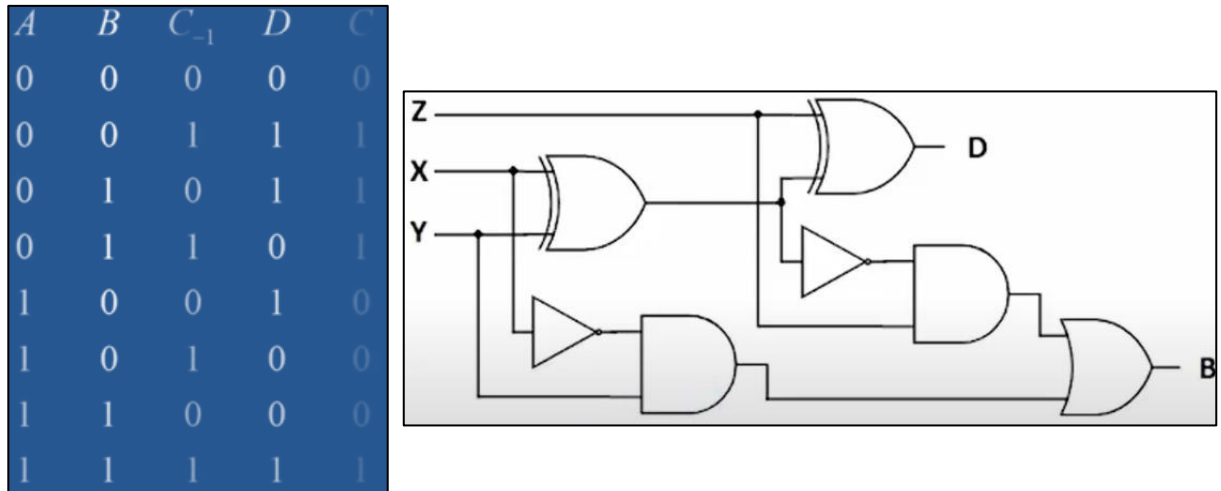


Figure: 9 TrueTable & Circuit 1BitSubtractor

Understanding this part is possibly create the true table for 8Bit Subtractor that allow do the same logic operation but with 8 circuits that represent this part and the Overflow when the value surpasses the max value for the representation.

A (dec)	A (bin)	B (dec)	B (bin)	A - B (bin)	A - B (dec)	Overflow
0	00000000	0	00000000	00000000	0	0
0	00000000	1	00000001	11111111	-1	0
1	00000001	0	00000000	00000001	1	0
1	00000001	1	00000001	00000000	0	0
2	00000010	0	00000000	00000010	2	0
2	00000010	1	00000001	00000001	1	0
3	00000011	0	00000000	00000011	3	0
3	00000011	1	00000001	00000010	2	0
4	00000100	0	00000000	00000100	4	0
4	00000100	1	00000001	00000011	3	0
5	00000101	0	00000000	00000101	5	0
5	00000101	1	00000001	00000100	4	0
6	00000110	0	00000000	00000110	6	0
6	00000110	1	00000001	00000101	5	0
7	00000111	0	00000000	00000111	7	0
7	00000111	1	00000001	00000110	6	0

Figure: 10 TrueTable 8Bit Subtractor

2.2.- Results

During the debugging tests, the values suggested in the requirements document were used. From the complete table, the following cases were considered:

- 3 cases where Overflow occurs
- 1 case where Overflow does not occur

A	B	A-B (expected)	Overflow
36 (8'b00100100)	-127 (8'b10000001)	-93 (8'b10100011)	1



Figure: 11 Test1

18 (8'b00010010)	-113 (8'b10001111)	-125 (8'b10000011)	1
---------------------	-----------------------	-----------------------	---



Figure: 12 Test2

A	B	R	Overflow
(127) = 01111111	(-127) = 01111112	(254)=1 11111110	1



Figure: 13 Test3

9 (8'b00001001)	99 (8'b01100011)	-90 (8'b10100110)	0
--------------------	---------------------	----------------------	---



Figure: 14 Test4

These are some of the cases tested during debugging to ensure the correct functioning of the arithmetic logic operations and to force the carry value. In this case, the maximum representable values in 8 bits range from -128 to 127. Any value beyond this range will cause an overflow. For these cases the representation can be represented in these values spaces.

2.3.- GitHub Repository

A branch was created in the course repository corresponding to Lab5_Subtractor.

https://github.com/CeicGitHub/Fundamentals_Of_Digital_Design_FPGA/tree/QuartusLab5_Subtractor

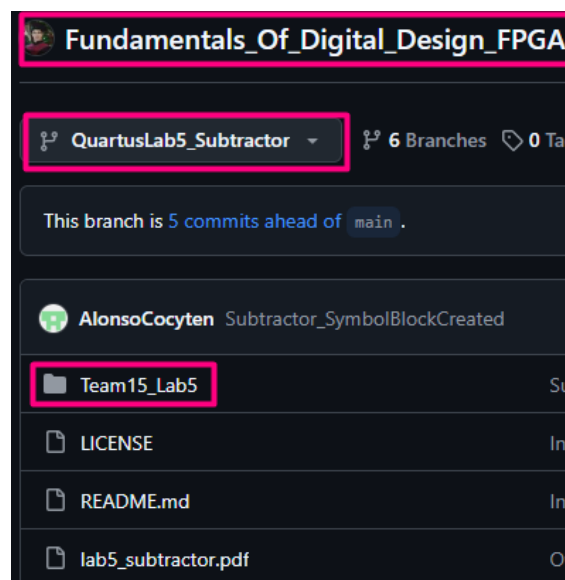


Figure: 15 GitHub Repository

3.- CHALLENGUES/ISSUES

Alonso Emmanuel López Macías

In this practice, I personally found it somewhat challenging to understand the operation of the subtractor, since it not only involved understanding it separately, but also required understanding how a half-adder and a full-adder work. These components give us the possibility, once their logic is designed or the circuit is built, to integrate them and implement an N-bit subtractor. This was a challenge because it required practicing that logic and also understanding its implementation in Quartus.

Cesar Eduardo Inda Cenicerros

One of the biggest challenges was being able to consider or interpret the logic for overflow. In my case, this occurred when adding an element called LPM_INV, which inverts the inputs of a buffer without needing to generate a combined negation. It became complicated because we realized that we were getting the value of the negated bit array in an XOR gate, but implementing incorrect logic, since we were not considering an important factor: we actually needed to use an XNOR gate. This gate would allow us to obtain the desired logic with the inverted value. Another issue was the Cin values specifically, assigning it a constant value of +1 using a VCC.

4.- CONCLUSIONS

Alonso Emmanuel López Macías

This practice helped me gain a deeper understanding of everything needed to implement a binary subtractor. It also allowed me to practice more in Quartus, particularly the part involving what is called modularization—converting our circuit into a symbol or function block. This greatly simplifies many processes in any other project or logic that needs to be implemented.

Cesar Eduardo Inda Cenicerros

I learned a lot from this lab. I understood how to create BSF elements or symbols to generate my own gates or library components and use them in any project. This greatly simplifies the process of creating logic gates and also makes it easier to share certain elements between projects by reusing these symbol libraries. I gained a deeper understanding of the functionality of the subtractor, including elements like overflow, which can even be used in an embedded system to trigger a flag in a process or task.